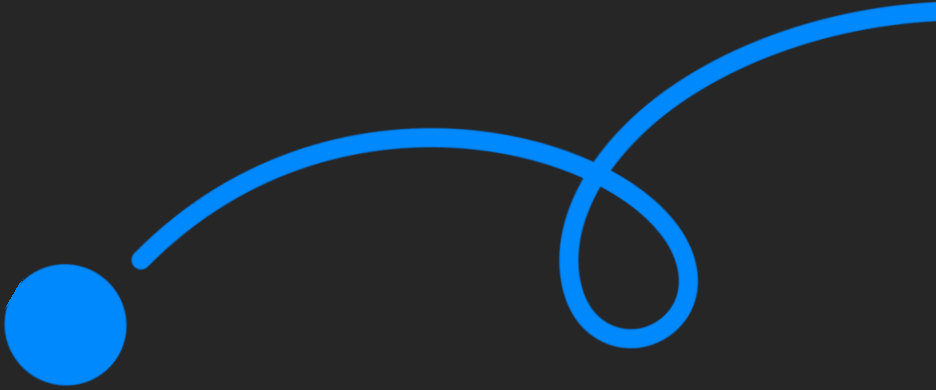




코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

김동현

최민우

최강주

지난 세션은 어떠셨나요.

- 1 지난 주차에 어떤걸 공부했었는지 짧게 리뷰 해봐요.
- 2 복습을 진행하면서 궁금한 점이 있다면 같이 토의해봐요.
- 3 과제를 진행하면서 어려웠던 점을 알려주세요.
- 4 멘티 여러분이 작성한 과제를 다같이 살펴보도록 해요.

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

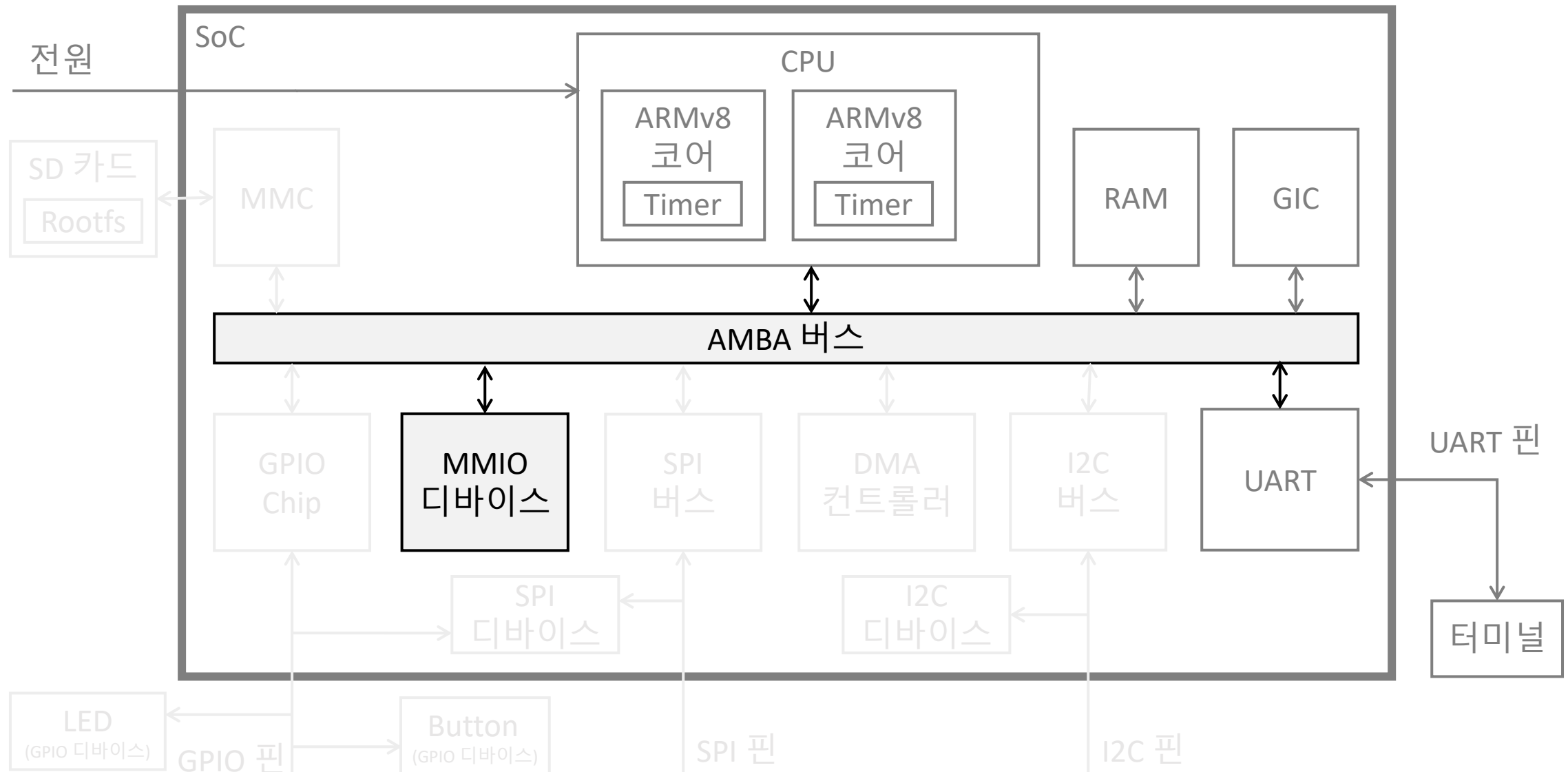
2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

이번 수업에서 진행할 내용



01 AMBA 버스 이해하기

02 MMIO 개념 파악하기

03 인터럽트와 폴링 개념 파악하기

04 새로운 MMIO 하드웨어 만들기

05 MMIO 하드웨어 드라이버 만들기

Chapter. 1

AMBA 버스 이해하기

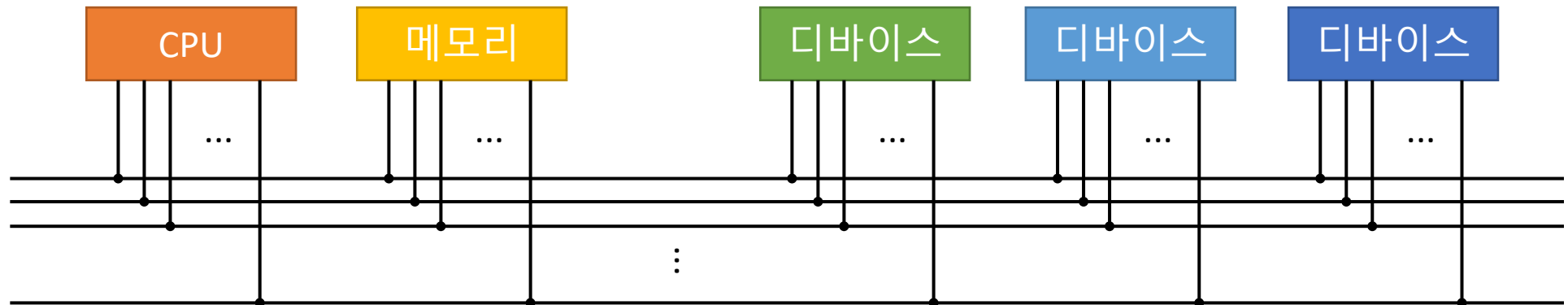
버스(Bus) 란?

CPU, 메모리, 디바이스간의 통신을 위한 공유된 통로

- 물리적으로는 수십 개의 전선으로 구성

디바이스의 개수가 몇 개이든 상관없이 버스에 연결될 수 있음

- 확장하기 좋은 구조, 단 버스를 충돌 없이 사용하기 위한 중재 매커니즘이 필요



AMBA (Advanced Microcontroller Bus Architecture) 버스란?

ARM사에서 개발한 SoC 내에서 사용하기 위해 개발한 버스

로열티가 없으며, 잘 문서화되어 외부에 공개되어 있으므로 누구라도 사용 가능

- AMBA 규격만 잘 지킨다면 어떠한 디바이스라도 SoC 안에 쉽게 추가될 수 있음
- 버스 구조라 확장이 편리하므로 반도체 제조사는 자신만의 특색 있는 SoC 개발 가능
- <https://developer.arm.com/documentation/ih0011/a>

ARM은 AMBA 버스를 사용해 다양한 디바이스에 대한 Eco-system을 구축해 옴

3가지 타입의 라인들로 구성 : 주소, 제어, 데이터

AMBA 버스의 구조

버스는 공유된 통로이므로 가장 느린 장치가 버스의 성능을 좌우하게 됨
연결되는 장치의 종류에 따라 버스는 여러 종류로 나뉨

- AHB (Advanced High Performance Bus) : CPU, 메모리, DMA 컨트롤러 등 고성능 요구하는 장치
- APB (Advanced Peripheral Bus) : 고성능을 요구하지 않는 주변 장치
- 브리지 (Bridge) : 성능이 다른 버스 사이를 중재해 주는 장치
- X86 계열 컴퓨터의 메인보드에서 사우스 브리지와 노스 브리지의 역할과 유사



✓ Peripheral 이란? CPU, 메모리 등 컴퓨터의 핵심 장치가 아닌 주변 장치를 의미합니다.

QEMU에서 AMBA 버스 사용하기

QEMU는 에뮬레이터이다 보니 AMBA 버스의 형태가 명확히 드러나진 않음

- ARM CPU 머신의 기본 시스템 버스가 AMBA 버스라고 가정됨
- QEMU에서 시스템 버스 사용하는 과정

- qdev_new로 새로운 디바이스를 생성
- SYS_BUS_DEVICE로 시스템 버스 디바이스로 변환
- 시스템버스 디바이스의 MMIO 영역과 인터럽트를 메모리와 GIC에 연결

```
DeviceState *dev = qdev_new(TYPE_PL011);
SysBusDevice *s = SYS_BUS_DEVICE(dev);
sysbus_realize_and_unref(s, &error_fatal);
memory_region_add_subregion(mem, base, sysbus_mmio_get_region(s, 0));
sysbus_connect_irq(s, 0, qdev_get_gpio_in(vms->gic, irq));
```

sysbus_create_simple을 사용하여 기본 시스템 버스 새로운 버스를 추가할 수 있음

➤ I2C와 SPI는 AMBA와는 다른 별개의 버스인데 이후 수업에서 자세히 살펴 볼 예정입니다.

Chapter. 2

MMIO 개념 파악하기

MMIO (Memory Mapped I/O) 란?

디바이스의 입출력 레지스터가 메모리 주소 점유하는 형태(메모리 맵)를 사용

- 점유하고 있는 주소를 사용하면 디바이스의 입출력 레지스터에 값 읽기/쓰기 가능

MMIO가 이루어지는 과정

1. CPU 명령어에서 메모리 주소에 읽기/쓰기 연산 실행
2. CPU에서는 시스템 버스의 각 라인으로 신호 전달
 - 주소라인으로 메모리주소를 제어라인으로 읽기/쓰기 여부를 전달
 - 쓰기라면 CPU에서 시스템 버스의 데이터라인으로 데이터도 전달
3. 디바이스는 주소 라인에 있는 주소가 자신에게 해당된다면, 제어라인 신호를 처리
 - 디바이스에서는 쓰기라면, 데이터라인으로 데이터를 받아 처리
 - 디바이스에서는 읽기라면, 데이터라인으로 데이터를 전달

PMIO (Port mapped I/O) vs MMIO

메모리 주소를 사용하는 것이 아닌 I/O 포트라는 별개의 주소를 사용

- I/O 포트에 읽거나 쓰기 위한 별개의 명령어가 요구됨
 - ARM에서는 사용하지 않음 – RISC 방식이기 때문 (ARM은 Advanced RISC Machine의 약자)
- 버스 구조 상으로도 I/O 포트를 위한 별개의 라인이 요구됨

X86에서는 PMIO와 MMIO 둘 다 사용함

- 기본적으로는 PMIO를 사용하고, MMIO는 PCI 버스를 위해 사용됨
- PMIO에 사용되는 어셈블리 명령어
 - IN <레지스터> <숫자> : 숫자에 해당하는 I/O 포트에서 CPU 레지스터로 값을 읽기
 - OUT <레지스터> <숫자> : CPU 레지스터의 값을 숫자에 해당하는 I/O 포트에 쓰기

RISC vs CISC

두 방식 모두 CPU의 명령어를 설계하는 방식

CISC (Complex Instruction Set Computer)

- 복잡하고 많은 종류의 명령어를 사용, 특히 명령어 각각의 길이가 다름
- X86 계열 CPU에서 사용되나 하위호환성을 위한 것임 (요즘에는 RISC 방식이 대세)
 - 최근에는 X86 계열은 기계어를 적은 종류의 마이크로 코드로 작게 나눈 뒤 내부적으로 RISC 방식 사용

RISC (Reduced Instruction Set Computer)

- 간단하고 적은 종류의 명령어를 사용, 명령어의 길이는 고정
- 명령어의 해석속도가 빠르고 파이프라이닝 구조에 강점
- ARM 계열 CPU에서 사용

일반적으로 사용되는 MMIO 레지스터

MMIO 레지스터를 어떻게 사용할지는 장치 설계자가 마음대로 결정할 수 있음
하나, 일반적으로 디바이스가 MMIO로 하는 일은 비슷하므로 아래의 패턴이 자주 사용

- Control Register : 각 비트를 통해 해당하는 기능을 활성화/비활성화
- Data Register : 데이터를 수신하거나 송신할 때 사용
- Flag Register : 각 비트를 통해 해당하는 상태를 나타냄
- Raw Interrupt Status Register : 각 비트를 통해 해당하는 인터럽트 발생 상태를 나타냄
- Interrupt Clear Register : 각 비트를 통해 해당하는 인터럽트 발생을 끄
- Interrupt Mask Set/Clear Register : 각 비트를 통해 해당하는 인터럽트 활성화/비활성화
- Masked Interrupt Status Register : 활성화된 인터럽트의 발생 상태를 나타냄



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

인터럽트 개념 이해하기

폴링 (Polling)

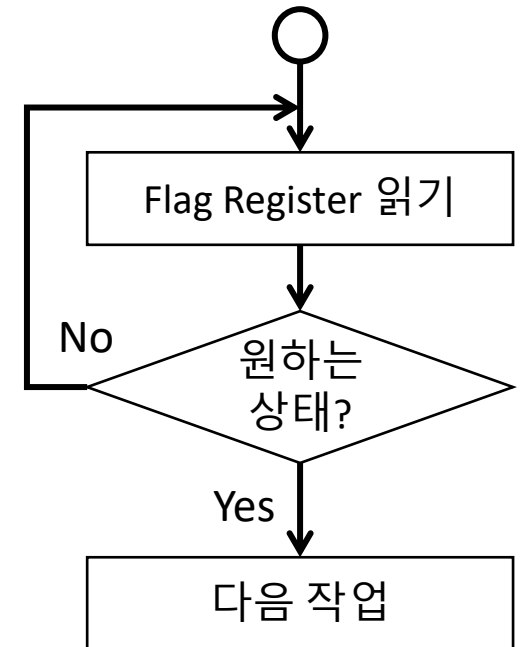
드라이버는 디바이스의 상태가 변할 때까지 기다려야 하는 경우가 많음

주기적으로 원하는 상태로 바뀔 때까지 체크하면서 기다리는 방식

- 보통 while문을 사용하고 조건문에 원하는 상태를 넣어 구현
- 일반적으로 Flag Register의 값을 지속적으로 확인하는 형태

매우 구현이 단순하나 기다리는 동안 CPU는 아무 작업도 처리하지 못함

- 오래 기다려야 하는 경우 성능 하락의 원인이 됨
- 하드웨어가 직접 상태의 변화를 알려주는 방식 -> 인터럽트



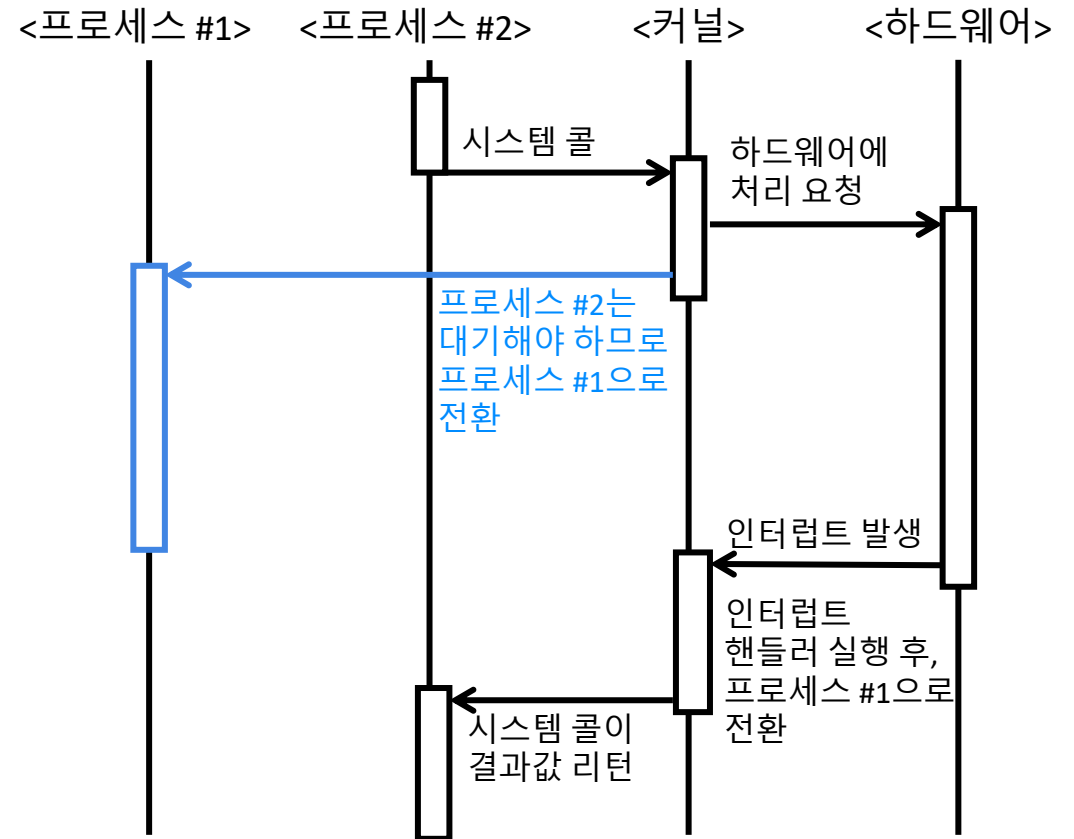
인터럽트 (Interrupt)

하드웨어가 CPU를 호출하는 것

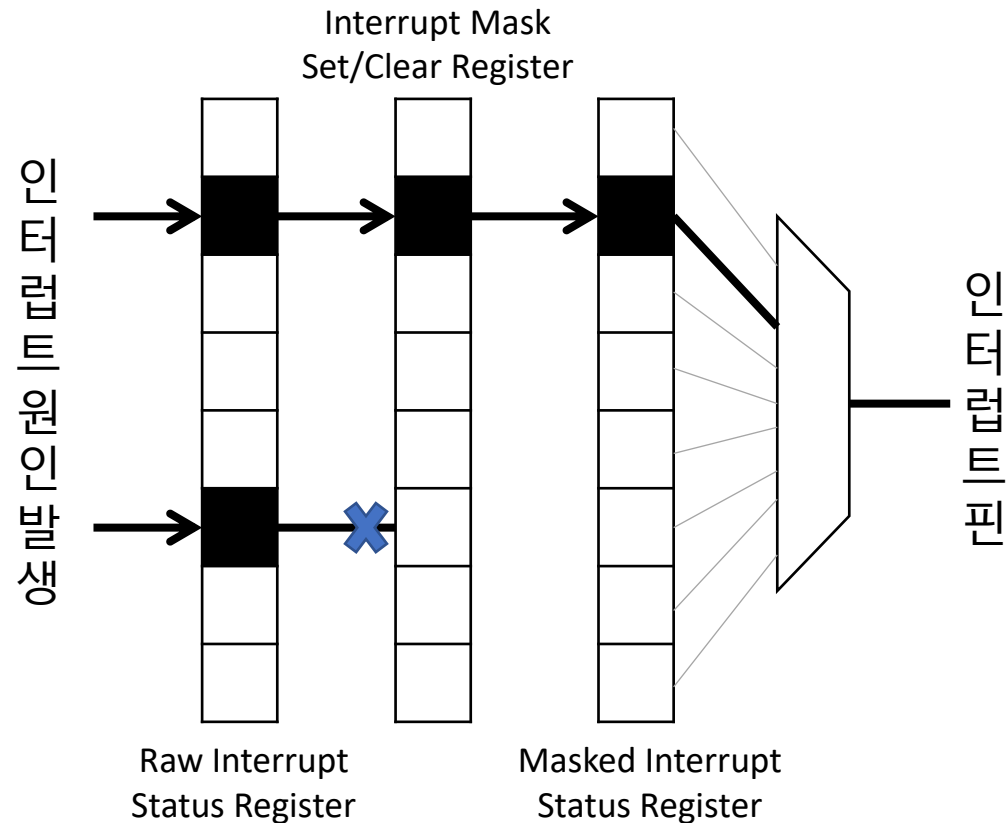
- 디바이스의 상태 변화를 알려줄 때 사용
 - 처리 결과나 예외적인 상황
- 인터럽트 덕분에 하드웨어를 무한히 기다리지 않고, 그 동안 다른 작업을 수행할 수 있음 (우측 파란색)

커널 모드의 인터럽트 핸들러가 바로 호출됨

- 인터럽트 핸들러가 지나치게 빈번하게 호출되면 다른 작업 처리에 방해가 받게 됨
- 인터럽트 핸들러 실행 중에는 CPU의 인터럽트 기능을 비활성화하여 방해 받지 않게 함



인터럽트 발생 흐름



1. 인터럽트는 여러 원인으로 발생할 수 있음
 - 원인은 Raw Interrupt Status Register에 기록됨
 - 동시에 여러 원인으로 발생하는 것도 가능
2. 각 원인에 대한 인터럽트는 선택적 활성화
 - 활성화되어 있지 않은 인터럽트는 인터럽트 발생에 영향을 주지 못함
3. 원인을 제거하거나 Interrupt Clear Register로 인터럽트를 꺼야 인터럽트가 발생하지 않음
 - 그렇지 않으면, 인터럽트 핀의 신호가 꺼지지 않아 인터럽트 핸들러가 무한 실행됨

인터럽트 사용시 주의 사항

인터럽트 핸들러가 실행되는 동안은 인터럽트가 발생하지 않음

- 타이머 인터럽트도 발생하지 않으므로 스케줄링이 동작하지 않음
 - 따라서, 인터럽트 핸들러가 너무 길게 되면, 시스템이 멈춘 것처럼 보이게 됨

뮤텍스나 스핀락을 사용은 최대한 지양해야 함

- 누군가가 뮤텍스를 점유하는 상태에서 인터럽트 핸들러가 그 뮤텍스를 기다리게 되면 인터럽트 핸들러에서는 스케줄링이 되지 않으므로 뮤텍스는 영원히 해제되지 않음

너무 빈번한 인터럽트 발생은 성능 저하를 야기함

- 불필요한 인터럽트가 발생되지 않도록 Interrupt Mask Set/Clear Register를 적절히 사용해야 함

짧게 기다려도 되는 경우는 인터럽트 대신 폴링을 사용하는 것이 좋음

Top half, Bottom half

일반적으로 디바이스 드라이버를 구현할 때, 인터럽트 처리를 나누어 구현

전반부 (Top half) : 인터럽트 핸들러의 구현

- 인터럽트가 발생하지 않는 구간
- 인터럽트 핀의 신호를 빨리 끄는 것이 목적, Interrupt Clear Register 사용
- Bottom half 처리를 세팅하고 종료

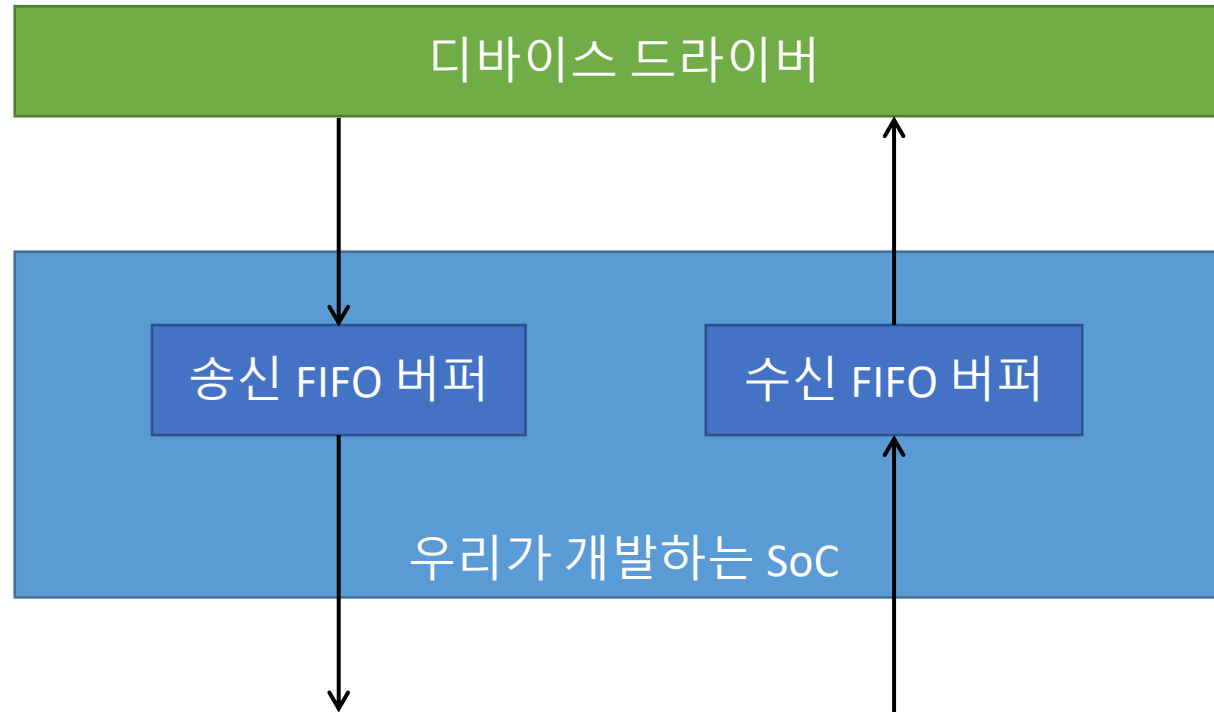
- 후반부 (Bottom half) : 하드웨어의 구체적인 처리 구현
- 인터럽트가 발생할 수 있는 구간, 오래 걸려도 알아서 스케줄링 됨
- 커널 내 별도의 스레드(Thread)로 수행됨
- 하드웨어로부터 데이터를 읽어오거나 쓰는 등 시간이 오래 걸리는 작업 수행

➤ 디바이스 드라이버를 만들면서 차근차근 알아보도록 합시다.

Chapter. 4

새로운 MMIO 하드웨어 만들기

수업에서 사용할 MMIO 하드웨어의 시나리오



- 하드웨어에 송신 FIFO 버퍼와 수신 FIFO 버퍼 존재
 - 디바이스 드라이버는 각각의 버퍼에 데이터를 송신 또는 수신하도록 구현
- QEMU에서는 어떤식으로 구현해 볼 수 있을까?

수업에서 만들 MMIO 하드웨어

QMP(QEMU Machine Protocol)로 문자열을 송수신 하는 디바이스

➤ QMP란? QEMU 위에서 동작하는 하드웨어의 속성을 설정하거나 받아오는 프로토콜

MMIO의 하드웨어에 data라는 이름의 문자열 값을 가질 수 있는 속성을 추가

- data 속성에 문자열을 set 하면 : 문자열을 하드웨어에 임시로 저장했다가 리눅스로 전달
- data 속성에서 문자열을 get 하면 : 리눅스에서 받아서 임시로 저장했던 문자열을 반환

문자열을 임시로 저장할 공간의 크기는 송/수신 각각 32byte

인터럽트 발생 : 리눅스가 받을 내용이 있을 때, 리눅스에서 보낸 내용이 다 처리 됐을 때

인터럽트 해제 : 리눅스가 받을 내용이 없을 때, 버퍼가 꽉 차서 리눅스에서 내용 못 보낼 때

QEMU에 새로운 하드웨어 추가하기

1. hw/misc 밑에 새로운 디렉토리를 만들고 우측과 같이 디렉토리를 hw/misc/meson.build의 맨 아래에 추가

```
subdir('comento')
```

<hw/misc/meson.build>

2. 새로 추가할 소스 코드가 빌드되도록 새 디렉토리에 meson.build를 아래와 같이 추가

```
softmmu_ss.add(when: 'CONFIG_COMENTO', if_true: files('mmio.c'))
```

<hw/misc/comento/meson.build>

QEMU에 새로운 MMIO 하드웨어 추가하기

1. 소스 코드를 아래와 같이 추가

<hw/misc/comento/mmio.c>

```
#include "qemu/osdep.h"
#include "hw/irq.h"
#include "hw/sysbus.h"
#include "hw/qdev-properties.h"

#define TYPE_MMIO_COMENTO "mmio-comento"

struct MMIO_COMENTO_State {
    SysBusDevice parent_obj;
    MemoryRegion iomem;
    qemu_irq irq;
};

OBJECT_DECLARE_SIMPLE_TYPE(MMIO_COMENTO_State, MMIO_COMENTO)

static uint64_t comento_mmio_read(void *opaque, hwaddr offset,
                                   unsigned size)
{
    return 0;
}

static void comento_mmio_write(void *opaque, hwaddr offset,
                               uint64_t value, unsigned size)
{
}
```

```
static const MemoryRegionOps comento_mmio_ops = {
    .read = comento_mmio_read,
    .write = comento_mmio_write,
    .endianness = DEVICE_NATIVE_ENDIAN,
};

static void comento_mmio_init(Object *obj)
{
    MMIO_COMENTO_State *s = MMIO_COMENTO(obj);
    SysBusDevice *dev = SYS_BUS_DEVICE(obj);

    memory_region_init_io(&s->iomem, obj, &comento_mmio_ops,
                          s, "mmio-comento", 0x1000);

    sysbus_init_mmio(dev, &s->iomem);
    sysbus_init_irq(dev, &s->irq);
}

static const TypeInfo comento_mmio_info = {
    .name = TYPE_MMIO_COMENTO,
    .parent = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(MMIO_COMENTO_State),
    .instance_init = comento_mmio_init,
};

static void comento_mmio_register_types(void)
{
    type_register_static(&comento_mmio_info);
}

type_init(comento_mmio_register_types)
```

AMBA ID 구성하기

AMBA 버스 하드웨어는 Peripheral ID와 Cell ID를 메모리 영역의 뒷부분에 표시해야 함

- 마지막 4byte로는 Cell ID, 그 앞의 4byte로는 Peripheral ID를 표시
- 리눅스에서 하드웨어를 자동으로 탐지하는데 사용됨

Cell ID : 하드웨어가 어떤 아키텍처를 사용하여 구현되었는지를 가리킴

- 우리가 사용할 CID는 AMBA_CID (0xB105F00D)

Peripheral ID : 하드웨어 자체의 ID를 나타냄

비트	설명
0 ~ 11	하드웨어의 모델 번호 (ARM PL011(UART)은 0x11)
12 ~ 19	하드웨어 제조사의 번호, (ARM은 0x41)
20 ~ 23	하드웨어 버전
24 ~ 31	하드웨어의 설정 상태를 나타내는 번호

MMIO 레지스터 설계하기 (1/3)

위치	이름	설명										
0x00	Data Register	읽기 : QMP로 받아 저장한 문자열을 받은 순서대로 1byte 반환										
		쓰기 : QMP로 전달하기 위해 저장하는 문자열 뒤에 1byte 저장										
0x04	Flag Register	읽기 : 현재 하드웨어의 버퍼 상태를 비트로 반환										
		<table><tr><th>비트</th><th>설명</th></tr><tr><td>0번</td><td>QMP에서 수신하는 버퍼 꽉 참</td></tr><tr><td>1번</td><td>QMP에서 수신하는 버퍼 텅 빈</td></tr><tr><td>2번</td><td>QMP로 보낼 문자열 버퍼 꽉 참</td></tr><tr><td>3번</td><td>QMP로 보낼 문자열 버퍼 텅 빈</td></tr></table>	비트	설명	0번	QMP에서 수신하는 버퍼 꽉 참	1번	QMP에서 수신하는 버퍼 텅 빈	2번	QMP로 보낼 문자열 버퍼 꽉 참	3번	QMP로 보낼 문자열 버퍼 텅 빈
		비트	설명									
		0번	QMP에서 수신하는 버퍼 꽉 참									
		1번	QMP에서 수신하는 버퍼 텅 빈									
		2번	QMP로 보낼 문자열 버퍼 꽉 참									
3번	QMP로 보낼 문자열 버퍼 텅 빈											

MMIO 레지스터 설계하기 (2/3)

위치	이름	설명
0x08	Raw Interrupt Status Register	읽기 : 인터럽트 원인의 충족 현황을 비트로 반환 (0 : 원인 미충족 / 1: 원인 충족, 인터럽트 발생 가능)
		비트 설명
		0번 QMP에서 데이터를 수신해, 버퍼에 데이터가 있음
		1번 QMP로 버퍼의 데이터를 전부 보내, 버퍼에 데이터 없음
0x0C	Interrupt Clear Register	쓰기 : 1로 지정한 비트의 인터럽트 원인을 미충족으로 변경
0x10	Interrupt Mask Set/Clear Register	읽기 : 인터럽트의 활성화 현황을 비트로 반환 (0 : 미활성화, 인터럽트 원인이 충족되더라도 인터럽트 미발생/ 1 : 활성화, 인터럽트 원인이 충족되면 바로 인터럽트 발생)
		쓰기 : 비트로 지정한 인터럽트를 활성화하거나 미활성화
0x14	Masked Interrupt Status Register	활성화된 인터럽트 중 충족된 인터럽트 원인을 비트로 반환

MMIO 레지스터 설계하기 (3/3)

위치	이름	설명
0x018 ~ 0xFDC	Reserved	사용되지 않음
0xFE0 ~ 0xFEC	Peripheral ID	읽기 : AMBA Peripheral ID 반환 - 모델 번호 : 0x001 - 제조사 번호 : 0xaa - 버전 번호 : 0x1 - 설정 번호 : 0x0
0xFE0 ~ 0xFEC	Cell ID	읽기 : AMBA Cell ID 반환

MMIO 레지스터 구성하기

<hw/misc/comento/mmio.c>

```
...
#define COMENTO_DR      0x00    /* Data register */
#define COMENTO_FR      0x04    /* Flag register */
#define COMENTO_RIS     0x08    /* Raw Interrupt Status Register */
#define COMENTO_ICR     0x0c    /* Interrupt Clear Register */
#define COMENTO_IMSC    0x10    /* Interrupt Mask Set/Clear Register */
#define COMENTO_MIS     0x14    /* Masked Interrupt Status Register */
#define COMENTO_FLAG_TXFE (1 << 3)
#define COMENTO_FLAG_TXFF (1 << 2)
#define COMENTO_FLAG_RXFE (1 << 1)
#define COMENTO_FLAG_RXFF (1 << 0)
// Peripheral ID - 0x001A_A001, Cell ID 0xB105_F00D
static const unsigned char comento_mmio_id[] = {0x01, 0xa0, 0x1a, 0x00,
                                                0x0d, 0xf0, 0x05, 0xb1};

struct MMIO_COMENTO_State {
...
    uint32_t flag, ris, imsc;
};
...
static uint64_t comento_mmio_read(void *opaque, hwaddr offset,
                                unsigned size)
{
    MMIO_COMENTO_State *s = (MMIO_COMENTO_State *)opaque;
    uint64_t r = 0;
    switch (offset) {
    case COMENTO_DR:
        /* TODO : implement this */
        break;
    case COMENTO_FR:
        r = s->flag;
        break;
```

```
    case COMENTO_RIS:
        r = s->ris;
        break;
    case COMENTO_IMSC:
        r = s->imsc;
        break;
    case COMENTO_MIS:
        r = s->ris & s->imsc;
        break;
    case 0xfe0 ... 0xfff:
        r = comento_mmio_id[(offset - 0xfe0) >> 2];
        break;
    }
    return r;
}

static void comento_mmio_write(void *opaque, hwaddr offset,
                               uint64_t value, unsigned size)
{
    MMIO_COMENTO_State *s = (MMIO_COMENTO_State *)opaque;
    switch (offset) {
    case COMENTO_DR:
        /* TODO : implement this */
        break;
    case COMENTO_ICR:
        s->ris ^= value;
        break;
    case COMENTO_IMSC:
        s->imsc = value;
        break;
    }
}
```

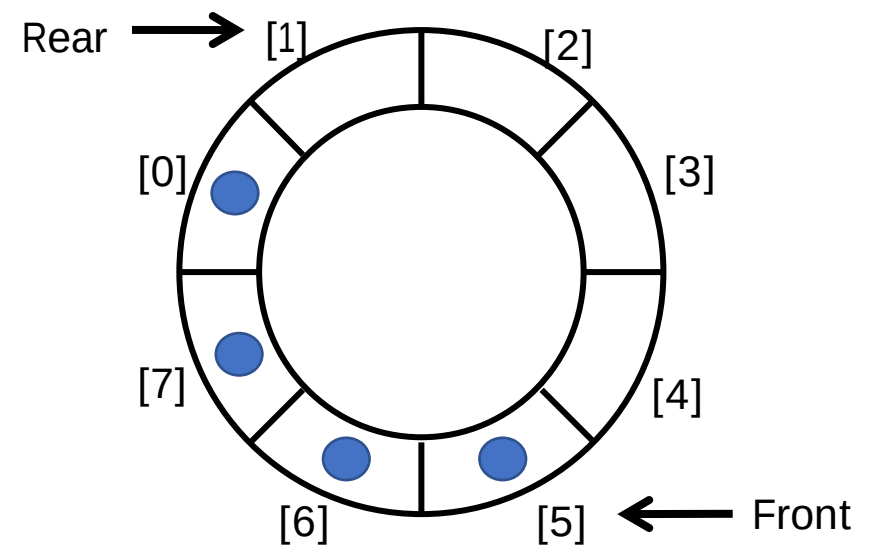
송/수신 FIFO 버퍼 설계하기

디바이스 내에는 데이터를 처리하기 전에 임시로 보관할 공간이 필요

- 일반적으로 디바이스의 데이터는 처음 들어간 것이 처음에 나와야 함 – FIFO (First-In First-Out)
- 이러한 공간을 FIFO 버퍼라고 부르며 링 버퍼(Ring buffer)의 형태로 구현됨

Front와 Rear를 관리하는 큐(Queue)형태의 자료구조

- Enqueue : Rear가 가리키고 있는 위치에 값을 넣고,
 $\text{Rear} = (\text{Rear} + 1) \% \text{QUEUE_SIZE}$
- Dequeue : Front가 가리키고 있는 위치 값을 반환하고,
 $\text{Front} = (\text{Front} + 1) \% \text{QUEUE_SIZE}$
- $\text{Front} == \text{Rear}$: 큐가 비어있는 상황
- $\text{Front} == (\text{Rear} + 1) \% \text{QUEUE_SIZE}$: 큐가 꽉 차있는 상황



FIFO 버퍼 구성하기

<hw/misc/comento/mmio.c>

```
...
#define COMENTO_FIFO_SIZE 32
struct fifo {
    int front, rear;
    uint8_t buf[COMENTO_FIFO_SIZE];
};
...
struct MMIO_COMENTO_State {
    ...
    struct fifo transmit, receive;
};
...
static void comento_fifo_enqueue(struct fifo *fifo, uint8_t data)
{
    fifo->buf[fifo->rear++] = data;
    fifo->rear %= COMENTO_FIFO_SIZE;
}
static uint8_t comento_fifo_dequeue(struct fifo *fifo)
{
    uint8_t ret = fifo->buf[fifo->front++];
    fifo->front %= COMENTO_FIFO_SIZE;
    return ret;
}
static bool comento_fifo_is_full(struct fifo *fifo)
{
    return fifo->front == ((fifo->rear + 1) % COMENTO_FIFO_SIZE);
}
static bool comento_fifo_is_empty(struct fifo *fifo)
{
    return fifo->front == fifo->rear;
}
```

```
static uint64_t comento_mmio_read(void *opaque, hwaddr offset,
                                   unsigned size)
...
    case COMENTO_DR:
        if (!comento_fifo_is_empty(&s->receive)) {
            r = comento_fifo_dequeue(&s->receive);
        }
        if (comento_fifo_is_empty(&s->receive)) {
            s->flag &= ~COMENTO_FLAG_RXFF;
            s->flag |= COMENTO_FLAG_RXFE;
        } // 데이터를 전부 읽어서 버퍼가 비었으므로 Flag Register 세팅
        break;
...
static void comento_mmio_write(void * opaque, hwaddr offset,
                                uint64_t value, unsigned size)
...
    case COMENTO_DR:
        if (!comento_fifo_is_full(&s->transmit)) {
            comento_fifo_enqueue(&s->transmit, value);
        }
        s->flag &= ~COMENTO_FLAG_TXFE; // 더 이상 비어있지 않으므로 세팅
        if (comento_fifo_is_full(&s->transmit)) {
            s->flag |= COMENTO_FLAG_TXFF;
        } // 데이터를 많이 보내서 버퍼가 꽉 찼으므로 Flag Register 세팅
        break;
...
static void comento_mmio_init(Object *obj)
...
    // 처음에는 송수신 버퍼 모두 비어있으므로 Flag Register 세팅
    s->flag = COMENTO_FLAG_TXFE | COMENTO_FLAG_RXFE;
...

```

QMP로 설정 가능한 속성 추가하기

<hw/misc/comento/mmio.c>

```
...
// data 속성의 값을 조회했을 때 호출되는 함수, 반환 값이 data 속성 값이 됨
static char *comento_get_data(Object *obj, Error **errp)
{
    MMIO_COMENTO_State *s = MMIO_COMENTO(obj);
    char *ret = g_malloc(COMENTO_FIFO_SIZE + 1);
    int i;

    for (i = 0; !comento_fifo_is_empty(&s->transmit); i++) {
        ret[i] = comento_fifo_dequeue(&s->transmit);
    } // 버퍼에 있는 내용을 모두 비움
    ret[i] = '\0';
    s->flag &= ~COMENTO_FLAG_TXFF; // 버퍼가 비었으므로
    s->flag |= COMENTO_FLAG_TXFE; // Flag Register에 업데이트 함

    return ret;
}
// data 속성에 값을 설정할 때 호출되는 함수, 파라미터로 설정할 값이 전달됨
static void comento_set_data(Object *obj, const char *value, Error **errp)
{
    MMIO_COMENTO_State *s = MMIO_COMENTO(obj);
    int count = strlen(value);
    int i;

    if (count == 0) {
        return;
    }
    for (i = 0; i < count && !comento_fifo_is_full(&s->receive); i++) {
        comento_fifo_enqueue(&s->receive, value[i]);
    }
}
```

```
s->flag &= ~COMENTO_FLAG_RXFE; // 버퍼에 데이터가 들어와서
// 더 이상 비어있지 않음을 Flag Register에 설정
if (comento_fifo_is_full(&s->receive)) {
    s->flag |= COMENTO_FLAG_RXFF;
} // 버퍼가 꽉찼다면 이를 Flag Register에 설정
}

...
static void comento_mmio_init(Object *obj)
{
    ...

    object_property_add_str(obj, "data", comento_get_data,
                             comento_set_data);
    ...
}
```

인터럽트 발생 구성하기

<hw/misc/comento/mmio.c>

```
...
#define COMENTO_INT_TX    (1 << 1)
#define COMENTO_INT_RX    (1 << 0)
...
static void comento_mmio_update(MMIO_COMENTO_State *s)
{
    uint32_t flags = s->ris & s->imsc;
    // 원인 총족(ris) & 활성화되어있는 인터럽트(mis) 가 하나라도 있다면
    // 인터럽트 발생!
    qemu_set_irq(s->irq, flags? 1 : 0);
}
...
static uint64_t comento_mmio_read(void *opaque, hwaddr offset,
                                unsigned size)
...
case COMENTO_DR:
...
    if (comento_fifo_is_empty(&s->receive)) {
        s->ris &= ~COMENTO_INT_RX;
    } // 리눅스가 받을게 없으면 인터럽트 해제
    comento_mmio_update(s);
    break;
...
static void comento_mmio_write(void *opaque, hwaddr offset,
                               uint64_t value, unsigned size)
...
case COMENTO_DR:
...
    if (comento_fifo_is_full(&s->transmit)) {
        s->ris &= ~COMENTO_INT_TX;
    } // 버퍼가 꽉차서 리눅스가 보낼 수 없다면 인터럽트 해제
```

```
comento_mmio_update(s);
break;
case COMENTO_ICR:
    s->ris ^= value;
    comento_mmio_update(s); // 인터럽트 발생하거나 해제
    break;
case COMENTO_IMSC:
    s->imsc = value;
    comento_mmio_update(s); // 인터럽트 발생하거나 해제
    break;
...
static char *comento_get_data(Object *obj, Error **errp)
{
    ...
    s->ris |= COMENTO_INT_TX; // 리눅스가 보낸 내용이 모두 처리되었으므로
    comento_mmio_update(s);   // 인터럽트 발생

    return ret;
}
...
static void comento_set_data(Object *obj, const char *value, Error **errp)
{
    ...
    s->ris |= COMENTO_INT_RX; // 리눅스가 받을게 생겼으므로 인터럽트 발생
    comento_mmio_update(s);
}
...
```

QEMU의 새 SoC에 MMIO 하드웨어 추가하기

```
#include "monitor/qdev.h"
...
#define COMENTO_IRQ_MMIO 1

#define COMENTO_BASE_MMIO 0x09011000
...
static void create_mmio(const ComentoMachineState *vms, MemoryRegion *mem)
{
    hwaddr base = COMENTO_BASE_MMIO;
    int irq = COMENTO_IRQ_MMIO;
    char name[] = "mmio-comento";
    DeviceState *dev = qdev_new(name);
    SysBusDevice *s = SYS_BUS_DEVICE(dev);

    // qmp에서 사용할 이름 지정
    // 이름을 지정하지 않으면 /machine/unattached/device[번호]로
    // 사용해야 하며, 수동으로 번호를 찾아야 함
    qdev_set_id(dev, name, NULL);

    sysbus_realize_and_unref(s, &error_fatal);
    memory_region_add_subregion(mem, base, sysbus_mmio_get_region(s, 0));
    sysbus_connect_irq(s, 0, qdev_get_gpio_in(vms->gic, irq));
}
...
static void machcomento_init(MachineState *machine)
...
    create_mmio(vms, sysmem);
}
```

<hw/arm/comento.c>



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 5

MMIO 하드웨어 드라이버 만들기

디바이스 트리에 MMIO 하드웨어 추가하기

```
mmio@9020000 {  
    compatible = "comento,mmio", "arm,primecell";  
  
    interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;  
  
    reg = <0x00 0x9011000 0x00 0x1000>;  
  
    clock-names = "apb_pclk";  
    clocks = <&clock>;  
};
```

<arch/arm64/boot/dts/comento/comento.dts>

- AMBA 디바이스로 탐지되기 위해 compatible에 반드시 "arm,primecell" 추가 해야 함
- 인터럽트는 SPI로 1번 사용
- 메모리 영역 주소 추가
- AMBA 프로토콜을 사용하기 위해 필요한 Clock 추가

새로운 드라이버 추가하기

```
config COMENTO_DRIVER
    bool "Comento SoC device drivers"
    help
        This enables device drivers for Comento SoC
```

<drivers/comento/Kconfig>

1. drivers 밑에 새로운 디렉토리를 만들고 그 디렉토리에 Kconfig를 위와 같이 추가

2. 추가한 디렉토리에 Makefile을 우측과 같이 추가

```
obj-y += mmio.o
```

<drivers/comento/Makefile>

3. drivers/Kconfig에 새로 추가한 Kconfig를 우측과 같이 추가

```
source "drivers/comento/Kconfig"
```

<drivers/Kconfig>

4. drivers/Makefile에 새로 추가한 Makefile을 우측과 같이 추가

```
obj-$(CONFIG_COMENTO_DRIVER) += comento/
```

<drivers/Makefile>

5. 커널 빌드할 때 추가했던 defconfig에 추가한 config를 아래와 같이 추가

```
CONFIG_COMENTO_DRIVER=y <arch/arm64/configs/comento_defconfig>
```

6. 수정한 defconfig를 make comento_defconfig 명령어로 적용

AMBA 드라이버 추가하기

```
#include <linux/module.h>
#include <linux/interrupt.h>
#include <linux/hashtable.h>
#include <linux/pm_wakeirq.h>
#include <linux/amba/bus.h>

static const struct amba_id comento_mmio_ids[] = {
    {
        .id = 0x000aa001,
        .mask = 0x000fffff,
        .data = NULL,
    },
    {0, 0},
};

MODULE_DEVICE_TABLE(amba, comento_mmio_ids);

static struct amba_driver comento_mmio_driver = {
    .drv = {
        .name = "mmio-comento",
    },
    .id_table = comento_mmio_ids,
    .probe = comento_mmio_probe,
    .remove = comento_mmio_remove,
};

module_amba_driver(comento_mmio_driver);

MODULE_AUTHOR("DDunAnt<ddunant@comento.com>");
MODULE_DESCRIPTION("ARM AMBA Comento MMIO Driver");
MODULE_LICENSE("GPL");
```

- AMBA 디바이스 탐지를 위해 Peripheral ID를 명시
- 하드웨어 버전이나 설정 ID가 여러 가지일 수 있으므로 mask로 탐지할 때 유효한 비트지정
- AMBA 버스 드라이버에서 ID를 참조할 수 있도록 등록
- 드라이버를 나타내는 이름, ID 명시
- 디바이스를 위해 호출될 함수 명시
- probe : 디바이스가 탐지 되었을 때
- remove : 드라이버를 등록 해제 할 때

<drivers/comento/mmio.c>

디바이스가 탐지되었을 때 구현하기 (1/3)

```
...
struct comento_mmio_device {
    struct device *dev;
    void __iomem *base;
    rwlock_t lock;
    unsigned int minor;
    struct hlist_node hash;
};

static unsigned int comento_mmio_major;
static unsigned int comento_mmio_minor;
static struct class *comento_mmio_class;

static DEFINE_HASHTABLE(comento_mmio_table, 4);

static struct comento_mmio_device *comento_mmio_get_device(
    unsigned int minor)
{
    struct comento_mmio_device *node;
    hash_for_each_possible(comento_mmio_table, node, hash, minor) {
        if (node->minor == minor) {
            return node;
        }
    }

    return NULL;
}
```

<drivers/comento/mmio.c>

- 드라이버에서 사용하는 데이터를 구조체로 정의
- 디바이스를 등록할 때 사용할 주번호, 부번호, 클래스
- 주번호는 최초로 디바이스 하나가 탐지되었을 때 register_chrdev로 생성
- 부번호는 0부터 1씩 증가하며 할당
- 클래스는 uevent를 위해 사용
 - Uevent는 /dev에 디바이스 노드를 자동으로 만들어 줌
- file_operations에서 구현하는 함수들에서 데이터를 사용하기 위해, 부번호로 검색하는 해시테이블 추가
- 해당 함수들에서는 file 구조체를 사용하여 부번호를 쉽게 얻을 수 있음

디바이스가 탐지되었을 때 구현하기 (2/3)

<drivers/comento/mmio.c>

```
static int comento_mmio_probe(struct amba_device *adev,
                             const struct amba_id *id)
{
    int ret;
    struct comento_mmio_device *cmdev;

    ret = amba_request_regions(adev, NULL); // 사용할 물리주소 미리 요청
    if (ret) {
        goto err_req;
    }
    // 주 번호를 할당 받은 적이 없다면, register_chrdev로 할당
    if (!comento_mmio_major) {
        ret = register_chrdev(0, "mmio-comento", &comento_mmio_fops);
        if (ret < 0) {
            goto out;
        }
        comento_mmio_major = ret;
    }
    // 클래스를 할당 받은 적이 없다면 새로 추가, Uevent를 위해 사용
    if (!comento_mmio_class) {
        comento_mmio_class = class_create("comento");
        if (IS_ERR(comento_mmio_class)) {
            ret = PTR_ERR(comento_mmio_class);
            comento_mmio_class = 0;
            goto out;
        }
    }
    // dev_kzalloc을 사용하면 디바이스가 등록 해제 될 때 알아서 free 됨
    cmdev = devm_kzalloc(&adev->dev, sizeof(struct comento_mmio_device),
                        GFP_KERNEL);
```

```
    if (!cmdev) {
        ret = -ENOMEM;
        goto out;
    }
    // 디바이스 트리로 명시된 메모리 영역은 물리 주소이므로
    // 주소공간에서 바로 사용이 불가능함
    // 해당 메모리 영역을 주소공간에 매핑하고 가상주소 반환
    cmdev->base = devm_ioremap(&adev->dev, adev->res.start,
                             resource_size(&adev->res));
    if (!cmdev->base) {
        ret = -ENOMEM;
        goto out;
    }
    // Uevent를 사용하여 /dev 밑에 새로운 디바이스 노드를 추가
    cmdev->dev = device_create(comento_mmio_class, &adev->dev,
                              MKDEV(comento_mmio_major, comento_mmio_minor), NULL,
                              "comento-mmio%d", comento_mmio_minor);
    if (IS_ERR(cmdev->dev)) {
        ret = PTR_ERR(cmdev->dev);
        cmdev->dev = NULL;
        goto out;
    }
    //다음 디바이스를 위해 부번호 증가
    cmdev->minor = comento_mmio_minor++;
    // 부번호를 키로 사용하는 해시테이블에 디바이스 데이터 추가
    hash_add(comento_mmio_table, &cmdev->hash, cmdev->minor);
    // 읽기와 쓰기를 위한 스핀락 초기화
    rwlock_init(&cmdev->lock);

    printk(KERN_INFO "Comento MMIO device driver\n");
```

디바이스가 탐지되었을 때 구현하기 (3/3)

```
amba_set_drvdata(adev, cmdev);

device_init_wakeup(&adev->dev, true);

// 너무 길어서 PPT에는 명시하지 않았으나
// COMENTO_* 매크로들은 QEMU에서 했던것 처럼
// drivers/comento/mmio.c에 정의해야 함
writel(COMENTO_INT_RX | COMENTO_INT_TX, cmdev->base + COMENTO_IMSC);

ret = devm_request_irq(&adev->dev, adev->irq[0],
                      comento_mmio_interrupt,
                      0, "mmio-comento", cmdev);

if (ret) {
    goto out;
}

dev_pm_set_wake_irq(&adev->dev, adev->irq[0]);

return 0;
out:
amba_release_regions(adev);
err_req:

return ret;
}
```

<drivers/comento/mmio.c>

- 데이터를 amba_device에도 등록
- 현재 디바이스가 깨어 있음을 알림
- RX, TX 인터럽트를 모두 받도록 Interrupt Mask Set/Clear Register 설정
- 디바이스 트리로 명시된 인터럽트의 핸들러 등록
- 디바이스가 깨어있으므로 인터럽트를 받을수 있는 상황임을 알림

디바이스가 제거 될 때 부분 구현하기

```
static void comento_mmio_remove(struct amba_device *adev)
{
    struct comento_mmio_device *cmdev = dev_get_drvdata(&adev->dev);

    if (cmdev->dev) {
        device_destroy(comento_mmio_class, cmdev->dev->devt);
        hash_del(&cmdev->hash);

        device_init_wakeup(&adev->dev, false);
        dev_pm_clear_wake_irq(&adev->dev);
    }

    amba_release_regions(adev);
}
```

- 만들어 주었던 디바이스 노드 제거
- 사용하던 인터럽트 끄기

<drivers/comento/mmio.c>

송신 부분 구현하기

```
static ssize_t comento_mmio_write(struct file *fp, const char __user *buf,
                                size_t len, loff_t *ppos)
{
    struct comento_mmio_device *cmdev;
    ssize_t written_bytes = 0, bytes;
    char mmio_buf[COMENTO_FIFO_SIZE];
    int i;

    cmdev = comento_mmio_get_device(iminor(fp->f_inode));
    write_lock(&cmdev->lock);

    bytes = MIN(len, sizeof(mmio_buf));
    bytes -= copy_from_user(mmio_buf, buf, bytes);
    for (i = 0; i < bytes; i++) {
        if (readl(cmdev->base + COMENTO_FR) & COMENTO_FLAG_TXFF) {
            break;
        }
        writel(mmio_buf[i], cmdev->base + COMENTO_DR);
    }
    written_bytes += i;
    *ppos += written_bytes;

    write_unlock(&cmdev->lock);
    return written_bytes;
}

static struct file_operations comento_mmio_fops = {
    .read = comento_mmio_read,
    .write = comento_mmio_write,
};
```

가장 간단하게 생각해 볼 수 있는 구현 방법

1. 커널에 FIFO 크기인 32바이트의 버퍼를 두고, copy_from_user를 사용하여 사용자 공간의 데이터를 커널 공간으로 복사
2. 복사 받은 버퍼를 순회하면서 Data register에 1바이트씩 값을 쓰기
3. 중간에 버퍼가 가득 찼다면, 값 쓰는 것을 중단하고 성공한 바이트 수 반환

사용자 공간에서는 목표한 바이트 수를 모두 보내지 못했으므로 계속 시도하게 됨

- 무한 루프를 돌면서 마치 폴링처럼 시도하게 되므로 CPU 자원이 낭비됨
- 인터럽트를 사용해서 알려줄 수는 없을까?

<drivers/comento/mmio.c>

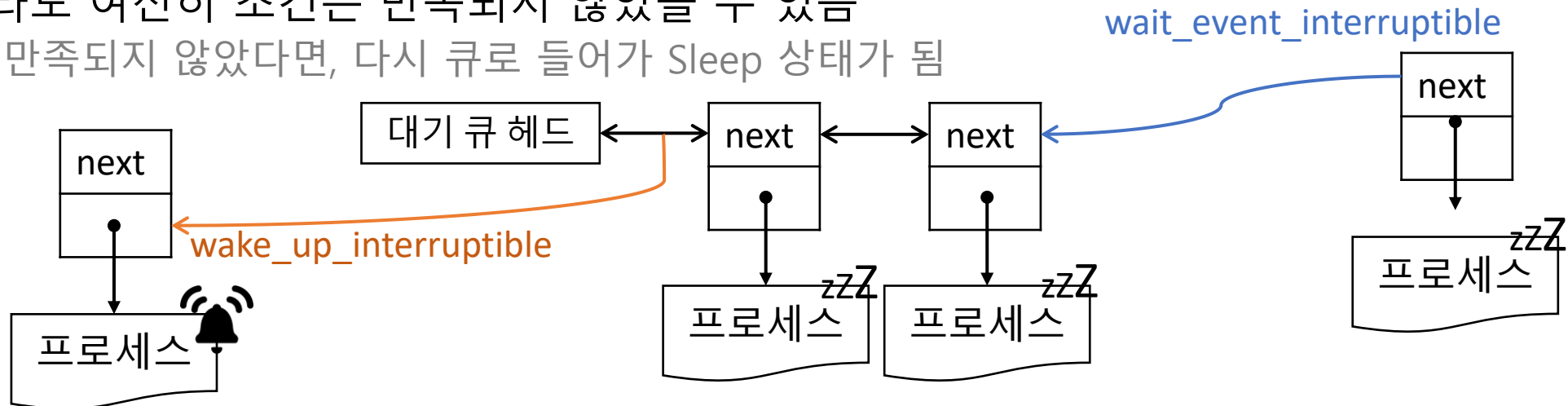
대기 큐 (Waiting queue)

원하는 조건이 만족하지 않았을 경우, 큐에 현재 프로세스를 넣고 Sleep 상태가 됨

- Sleep 상태란? 현재 프로세스가 실행되어도 무의미하다고 판단된 상태
 - Sleep 상태가 되자마자 다른 프로세스로 전환 됨
 - Sleep 상태의 프로세스는 누군가가 깨워주지 않는다면 스케줄링 되지 않음

깨어나더라도 여전히 조건은 만족되지 않았을 수 있음

- 조건이 만족되지 않았다면, 다시 큐로 들어가 Sleep 상태가 됨



송신 부분을 대기 큐로 개선하기

<drivers/comento/mmio.c>

```
...
#define MIN(a,b) ((a)<(b) ? (a):(b))

struct comento_mmio_device {
...
    wait_queue_head_t tx_wq; // 대기 큐의 헤드를 선언
...
};
...
static ssize_t comento_mmio_write(struct file *fp, const char __user *buf,
                                size_t len, loff_t *ppos)
{
...
    write_lock(&cmdev->lock);
    while (len > 0) {
        bytes = MIN(len, sizeof(mmio_buf));
        bytes -= copy_from_user(mmio_buf, buf, bytes);
        for (i = 0; i < bytes; i++) {
            wait_event_interruptible(cmdev->tx_wq, // 버퍼가 꽉찼다면
                                   !(readl(cmdev->base + COMENTO_FR) & // 대기 큐 들어가
                                     COMENTO_FLAG_TXFF)); // 누군가가 깨울 때까지 대기
            writel(mmio_buf[i], cmdev->base + COMENTO_DR);
        }
        buf += bytes;
        len -= bytes;
        written_bytes += bytes;
    }
    *ppos += written_bytes;
    write_unlock(&cmdev->lock);
...
}
```

```
static irqreturn_t comento_mmio_interrupt(int irq, void *dev_id)
{
    struct comento_mmio_device *cmdev = dev_id;
    unsigned long mis;
    mis = readl(cmdev->base + COMENTO_MIS);

    // Masked Interrupt Register를 통해 인터럽트가 발생한 원인을 파악
    if (mis & COMENTO_INT_TX) {
        // 데이터 송신을 완전히 완료하기 전에는 인터럽트가
        // 계속 발생할 것이므로 인터럽트를 Interrupt Clear Register로 끄
        writel(COMENTO_INT_TX, cmdev->base + COMENTO_ICR);
        // 대기큐에서 이벤트를 기다리는 모든 프로세스를 깨움
        wake_up_interruptible(&cmdev->tx_wq);
        // 인터럽트가 성공적으로 처리되었음을 반환
        return IRQ_HANDLED;
    }
    return IRQ_NONE;
}

...

static int comento_mmio_probe(struct amba_device *adev,
                             const struct amba_id *id)
{
...
    // 인터럽트 초기화하기 전에 대기큐 먼저 초기화
    init_waitqueue_head(&cmdev->tx_wq);

    writel(COMENTO_INT_RX | COMENTO_INT_TX, cmdev->base + COMENTO_IMSC);
...
}
```

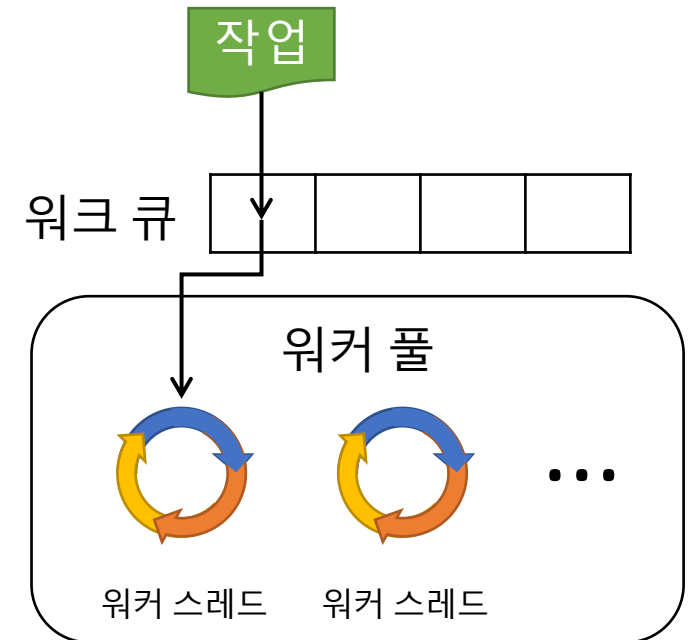
워크 큐 (Work Queue)

인터럽트 핸들러에서는 시간이 오래 걸리는 작업을 하면 안됨

- 인터럽트 핸들러는 Top-half로써, 인터럽트 원인을 확인하고 인터럽트를 해제하는 정도만 수행
- 특히, 데이터 수신은 오래 걸릴 수 있으므로, Bottom-half에서 작업해야 함

Bottom-half를 수행하기 위해 워크 큐가 도입됨

- 스레드 풀(Thread Pool)과 같은 방식으로 동작함
 1. 워커 스레드 (Worker Thread)는 부팅 될 때 생성
 1. 생성된 워커 스레드는 워커 풀에서 관리함
 2. 실행할 작업이 없으면 워커 스레드는 Sleep 상태로 들어감
 3. 누군가가 `schedule_work()`로 워크 큐에 원하는 작업을 넣음
 4. 워커 풀은 워크 큐에서 작업을 dequeue함
 5. 워커 풀은 Dequeue한 작업을 실행할 워커 스레드 하나를 깨움
 6. 깨어난 워커 스레드는 해당 작업 실행



수신 부분 구현하기

<drivers/comento/mmio.c>

```
...
struct comento_mmio_device {
...
    struct work_struct rx_work; // rx_buf 등 데이터 찾기 쉽게 하기 위해
    char rx_buf[32];           // 같은 데이터 구조체 내 work_struct 선언
    int rx_size;
...
};
...
// 단순히 rx_buf에서 데이터를 사용자 공간으로 전달하는 것이 전부
static ssize_t comento_mmio_read(struct file *fp, char __user *buf,
                                size_t len, loff_t *ppos)
{
    struct comento_mmio_device *cmdev;
    ssize_t read_bytes = 0;

    cmdev = comento_mmio_get_device(iminor(fp->f_inode));

    read_lock(&cmdev->lock);
    if (cmdev->rx_size <= len + *ppos) {
        len = cmdev->rx_size - *ppos;
    }
    read_bytes = len - copy_to_user(buf, cmdev->rx_buf + *ppos, len);
    *ppos += read_bytes;

    read_unlock(&cmdev->lock);

    return read_bytes;
}
...
```

```
static irqreturn_t comento_mmio_interrupt(int irq, void *dev_id)
{
...
    if (mis & COMENTO_INT_RX) { // 인터럽트 Interrupt Clear Register로 끄
        writel(COMENTO_INT_RX, cmdev->base + COMENTO_ICR);
        schedule_work(&cmdev->rx_work); // Bottom-half 실행 예약
        return IRQ_HANDLED;
    }
...
static void comento_mmio_rx_work_handler(struct work_struct *work)
{ // Bottom half 실행
    // work_struct 포인터로 work_struct가 속한 데이터 구조체를 얻어옴
    struct comento_mmio_device *cmdev = container_of(work,
        struct comento_mmio_device, rx_work);
    unsigned long flag, i;
    write_lock(&cmdev->lock); // rx_buf 다루는 동안은 모든 프로세스 배제
    for (i = 0, flag = readl(cmdev->base + COMENTO_FR);
        !(flag & COMENTO_FLAG_RXFE);
        flag = readl(cmdev->base + COMENTO_FR), i++) {
        cmdev->rx_buf[i] = readl(cmdev->base + COMENTO_DR);
    } // 버퍼가 텅 빌 때까지 Data Register에서 값을 읽어 rx_buf에 저장
    cmdev->rx_size = i;
    write_unlock(&cmdev->lock);
}
static int comento_mmio_probe(struct amba_device *adev,
                             const struct amba_id *id)
{
...
    // 인터럽트 초기화하기 전에 대기큐 먼저 초기화
    INIT_WORK(&cmdev->rx_work, comento_mmio_rx_work_handler);
...
}
```

QMP 스크립트 사용하기

QEMU로 실행할 때 QMP를 보내기 위한 소켓을 추가해야 함

- `-qmp unix:/tmp/qmp.sock,server,nowait`

QMP 스크립트로 수정 가능한 속성은 트리 구조로 사용가능

- `"/`가 최상위 노드, `/machine`부터 검색하면 현재 SoC의 모든 디바이스의 속성 사용 가능

QMP 스크립트를 사용하여 추가한 data 속성에 값을 읽거나 쓸 수 있음

- QMP 스크립트는 QEMU 코드의 `qemu/scripts/qmp`에 있음
- 목록 조회 : `qemu/scripts/qmp/qom-list --socket /tmp/qmp.sock /machine/peripheral/`
 - `qdev_set_id`로 ID 추가한 디바이스들의 ID 목록이 보임
 - `qdev_set_id`를 안 한 경우: `/machine/unattached/device[<번호>]`의 형태로 존재
- 속성 읽기 : `./qom-get --socket /tmp/qmp.sock /machine/peripheral/mmio-comento.data`
- 속성 쓰기 : `./qom-set --socket /tmp/qmp.sock /machine/peripheral/mmio-comento.data <문자열>`

QMP 스크립트 사용 실습

```
comento@comento:~/qemu/scripts/qmp$ ./qom-list --socket /tmp/qmp.sock /machine
/peripheral
type
mmio-comento/
comento@comento:~/qemu/scripts/qmp$ ./qom-list --socket /tmp/qmp.sock /machine
/peripheral/mmio-comento
type
@parent_bus/
realized
hotplugged
hotpluggable
@sysbus-irq[0]/
@sysbus-irq[1]/
@sysbus-irq[2]/
mmio-comento[0]/
data
comento@comento:~/qemu/scripts/qmp$ ./qom-set --socket /tmp/qmp.sock /machine/
peripheral/mmio-comento.data Hello world
{}
comento@comento:~/qemu/scripts/qmp$ ./qom-get --socket /tmp/qmp.sock /machine/
peripheral/mmio-comento.data
Nice to meet you
```

- SoC 내 ID가 있는 디바이스목록 확인

- data 속성이 존재하는 것 확인

- data 속성에 문자열을 설정함

- data 속성에 있는 문자열을 얻어옴

QEMU에서 디바이스와 드라이버 테스트 실습

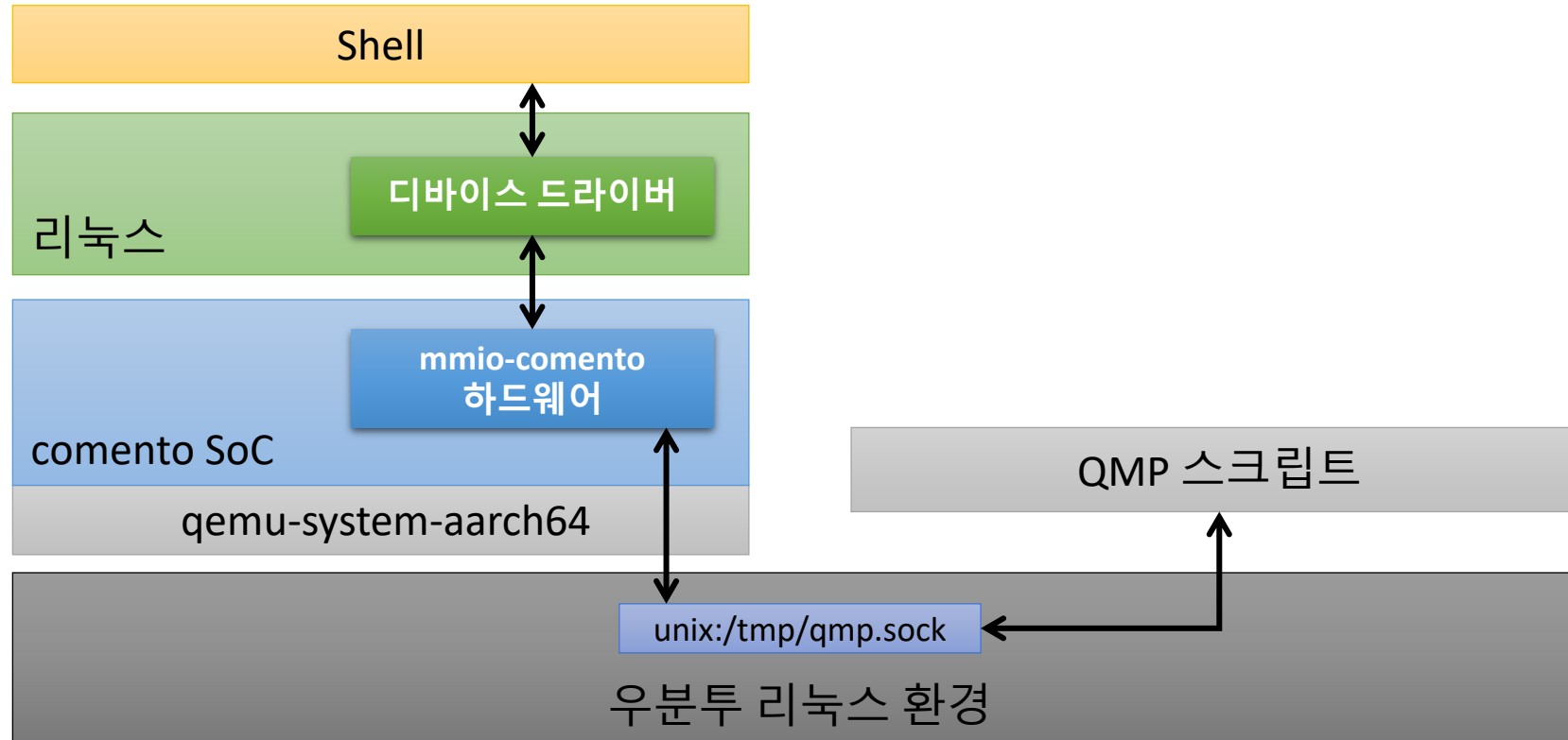
QEMU에서 리눅스 부팅중 MMIO 드라이버가 제대로 초기화 됐는지 확인

```
io scheduler kyber registered
io scheduler bfq registered
SPI driver spidev has no spi_device_id for spidev
i2c dev: i2c /dev entries driver
Comento MMIO device driver
NET: Registered PF_INET6 protocol family
Segment Routing with IPv6
```

/dev 밑에 생긴 comento-mmio0를 사용하여 데이터를 받아오거나 보내지는지 확인

```
Welcome to Buildroot
buildroot login: root
# cat /dev/comento-mmio0
Hello world#
# echo Nice to meet you > /dev/comento-mmio0
#
```

QEMU 사용시 자주 헷갈리시는 포인트



- QEMU와 소통하는 우분투 리눅스 != QEMU 위에서 구동되는 리눅스
- QEMU에서는 SoC 하드웨어 에뮬레이션을 구현
리눅스에서는 디바이스 드라이버를 구현



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

과제 안내

과제 의도

- 주어진 기준에 맞춰서 새로운 하드웨어를 QEMU에 구성해 본다.
- MMIO 하드웨어가 어떤 식으로 구성되는지 감을 잡을 수 있다.

필수 과제 내용

1. 수업에서 만들어 보았던 MMIO 하드웨어를 수정하여 Logging 디바이스로 만들어 본다.
 1. QMP 관련 구현은 사용하지 않는다.
 2. AMBA 하드웨어 ID는 0x002로 한다.
 3. 리눅스에서 데이터가 보내지면 /tmp/qemu-mmio.log 파일에 내용을 계속 기록한다.
 4. 리눅스에서 데이터를 수신하는 일은 없으므로 불필요한 구현은 제거한다.
2. 수업에서 만들어 보았던 드라이버를 사용하여 Logging 디바이스 드라이버를 만든다.
 1. 마찬가지로, 리눅스에서 데이터를 수신하는 일은 없으므로 불필요한 구현은 제거한다.
3. QEMU 디바이스 코드와 리눅스의 디바이스 드라이버, 디바이스 트리를 업로드하여 제출한다.

과제 안내

선택 과제 내용

1. Logging 디바이스에 로그 파일을 삭제하는 기능을 구현한다.
 1. Logging 디바이스에 0x18 주소를 사용하는 Register를 추가한다.
 - 해당 레지스터의 비트 0번에 1을 주면, /tmp/qemu-mmio.log를 삭제한다.
2. 드라이버에 로그 파일을 추가하는 기능을 ioctl을 사용하여 구현한다.
3. QEMU 디바이스 코드와 리눅스의 디바이스 드라이버를 업로드하여 제출한다.

기한 : 다음 주 월요일 오전 0시까지

다음 주 커리큘럼

DMA 파악하기

Scatter/ Gather 개념 이해하기

DMA 컨트롤러 추가하기

DMA API를 사용하여 드라이버 개선하기

MMC 하드웨어로 Rootfs 사용하기

